

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

Pertemuan 7

Penyimpanan Lokal dan Offline-first

Menyimpan laporan dan draf di perangkat dengan SQLDelight, supaya Laporkampus tetap berguna saat sinyal hilang

Muhammad Habib Algifari, S.Kom., M.T.I.

Tahun 2026



Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera

Tiga jawaban dari tiket keluar minggu lalu

Pertanyaan	Jawaban singkat
Apa arti kode status 404 dan 500, dan mana yang kesalahan di sisi klien?	404 data atau rute tidak ditemukan, sisi klien. 500 kesalahan di server
Kenapa data class butuh @Serializable, dan bagaimana dengan field tambahan?	Plugin butuh anotasi untuk membuat serializer. Field tambahan membuat error kecuali ignoreUnknownKeys
Kenapa keadaan layar ditulis sebagai sealed interface?	Mencegah keadaan mustahil seperti memuat sekaligus gagal, dan when memeriksa semua kemungkinan

Sinyal hilang, laporan ikut hilang

Di gedung kuliah sinyal sering hilang. Begitu offline, daftar laporan kosong. Draf yang sedang ditulis juga ikut hilang begitu aplikasi ditutup atau dimatikan sistem.

DISKUSI 2 MENIT

Bagaimana aplikasi tetap menampilkan data terakhir dan menyimpan draf, walau tidak ada internet?

LaporKampus

**Tidak bisa terhubung.
Periksa internet.**

Coba lagi

Daftar laporan: kosong

**Draf Pintu macet di GKU 2
R305 hilang setelah aplikasi
ditutup**

Saat pulang nanti, kalian bisa

1 **Memilih penyimpanan**
Membedakan kapan cukup preferensi, kapan butuh database, kapan berkas.

2 **Memakai SQLDelight**
Menulis tabel dan query di berkas .sq dan memakai kode Kotlin buatannya.

3 **Menerapkan offline-first**
Data terakhir tetap tampil saat offline dan draft form tidak hilang.

Capaian ini mendukung **CPMK0501**: menerapkan konsep pemrograman untuk pengembangan perangkat lunak, dalam bentuk aplikasi yang menyimpan data lokal dan tetap berfungsi tanpa internet.

Bagian 1

Pilihan penyimpanan

Nilai kecil, data terstruktur, dan berkas punya tempatnya masing-masing.

Tiga tempat menyimpan di perangkat

Pilihan	Cocok untuk	Contoh di LaporKampus
Preferensi key value DataStore Preferences	Beberapa nilai kecil, dibaca utuh	Tema gelap, nama pelapor, gedung favorit
Database SQLDelight	Banyak baris yang dicari, diurutkan, dan diubah sebagian	Daftar laporan, draf form
Berkas	Data besar yang tidak dicari isinya	Foto bukti laporan di minggu 8

Preferensi: satu kunci, satu nilai

commonMain/data/Pengaturan.kt

```
val TEMA_GELAP = booleanPreferencesKey("tema_gelap")

val temaGelap: Flow<Boolean> =
    datastore.data.map { it[TEMA_GELAP] ?: false }

suspend fun ubahTema(gelap: Boolean) {
    datastore.edit { it[TEMA_GELAP] = gelap }
}
```

Bagian	Gunanya
booleanPreferencesKey	Kunci bertipe, jadi nilainya pasti Boolean
data	Flow yang memancarkan nilai terbaru
edit	Menulis secara aman, suspend

Latihan 1 di repo meniru pola yang sama memakai tabel Preferensi di SQLDelight.

BANGUN BARENG

Dua ratus laporan disimpan di preferensi

Semuanya sebagai satu teks JSON di bawah kunci daftar. Apa yang terjadi saat status satu laporan berubah, atau saat kita ingin laporan di GKU 1 saja?

Bagian 2

SQLDelight

Kita menulis SQL, SQLDelight membuatkan kode Kotlin yang aman tipenya.

Memasang plugin dan driver

composeApp/build.gradle.kts, di dalam sourceSets

```
commonMain.dependencies {
    implementation(libs.sqldelight.coroutines)
}
androidMain.dependencies {
    implementation(libs.sqldelight.android.driver)
}
iosMain.dependencies {
    implementation(libs.sqldelight.native.driver)
}
desktopMain.dependencies {
    implementation(libs.sqldelight.sqlite.driver)
}
```

composeApp/build.gradle.kts

```
plugins {
    alias(libs.plugins.sqlDelight)
}
sqlDelight { databases {
    create("LaporKampusDb") {
        packageName.set(
            "id.ac.itera.laporkampus.db"
        )
    }
} }
```

Nama di `create("...")` menjadi nama kelas database. Semua library ada di grup `app.cash.sqlDelight`.

Berkas Laporan.sql

commonMain/sqlelight/.../Laporan.sql

```
CREATE TABLE LaporanEntity (  
    id INTEGER NOT NULL PRIMARY KEY,  
    judul TEXT NOT NULL,  
    lokasi TEXT NOT NULL,  
    status TEXT NOT NULL,  
    petugas TEXT  
);
```

Bagian	Artinya
semua:	Nama query, menjadi nama fungsi
?	Parameter yang diisi saat dipanggil
INSERT OR REPLACE	Tambah, atau timpa bila id sudah ada

Lanjutan berkas yang sama

semua:

```
SELECT * FROM LaporanEntity  
ORDER BY id DESC;
```

byId:

```
SELECT * FROM LaporanEntity WHERE id = ?;
```

simpan:

```
INSERT OR REPLACE INTO LaporanEntity  
VALUES (?, ?, ?, ?, ?);
```

hapusSemua:

```
DELETE FROM LaporanEntity;
```

BANGUN BARENG

Kolom petugas tanpa NOT NULL

Apa tipe petugas di kode Kotlin yang dibuatkan? Lalu, kalau di query simpan kita salah ketik nama tabel menjadi LaporanEntiti, kapan kesalahannya ketahuan?

Cek tebakanmu: dari .sq ke Kotlin

Dari Laporan.sql ke kode Kotlin

commonMain/sqlDelight/.../Laporan.sql

```
CREATE TABLE LaporanEntity (  
  id INTEGER NOT NULL PRIMARY KEY,  
  judul TEXT NOT NULL,  
  lokasi TEXT NOT NULL,  
  status TEXT NOT NULL,  
  petugas TEXT  
);  
  
semua:  
SELECT * FROM LaporanEntity  
ORDER BY id DESC;  
  
byId:  
SELECT * FROM LaporanEntity WHERE id = ?;  
  
simpan:  
INSERT OR REPLACE INTO LaporanEntity  
VALUES (?, ?, ?, ?, ?);  
  
hapusSemua:  
DELETE FROM LaporanEntity;
```

dibuatkan saat build

build/generated/sqlDelight

```
data class LaporanEntity(  
  val id: Long,  
  val judul: String,  
  val lokasi: String,  
  val status: String,  
  val petugas: String?  
)  
  
class LaporanQueries {  
  fun semua(): Query<LaporanEntity>  
  
  fun byId(id: Long): Query<LaporanEntity>  
  
  fun simpan(id: Long, judul: String,  
    lokasi: String, status: String,  
    petugas: String?)  
  
  fun hapusSemua()  
}
```

Kita menulis SQL. SQLDelight membuat kelas dan fungsi Kotlin yang aman tipenya.

COBA LANGSUNG



Scan atau klik untuk membaca panduan resmi SQLDelight multiplatform: plugin, berkas .sq, dan driver per platform.

[Getting Started with SQLite on Multiplatform](#)

Sumber: Dokumentasi SQLDelight. Bacaan lanjut:

[Coroutines, SQLDelight](#)

Animasi berputar saat presentasi. Di PDF yang tampil keadaan akhirnya.

Driver per platform lewat expect dan actual

commonMain/data/DriverFactory.kt

```
// commonMain
expect class DriverFactory {
    fun createDriver(): SqlDriver
}
```

androidMain/data/DriverFactory.android.kt

```
// androidMain
actual class DriverFactory(
    private val context: Context
) {
    actual fun createDriver(): SqlDriver =
        AndroidSqliteDriver(
            LaporkampusDb.Schema, context, "laporkampus.db")
}
```

iosMain/data/DriverFactory.ios.kt

```
// iosMain
actual class DriverFactory {
    actual fun createDriver(): SqlDriver =
        NativeSqliteDriver(
            LaporkampusDb.Schema, "laporkampus.db")
}
```

desktopMain/data/DriverFactory.desktop.kt

```
// desktopMain
actual class DriverFactory {
    actual fun createDriver(): SqlDriver =
        JdbcSqliteDriver("jdbc:sqlite:laporkampus.db",
            Properties(), LaporkampusDb.Schema)
}
```

Membuat database dan memanggil query

commonMain/App.kt

```
val db = LaporKampusDb(driverFactory.createDriver())
val q = db.laporanQueries

q.simpan(7, "AC mati", "GKU 1 R201", "Diproses", "Pak Joko")
q.simpan(8, "Kursi patah", "GKU 2 R305", "Baru", null)

val semua: List<LaporanEntity> = q.semua().executeAsList()
val satu: LaporanEntity? = q.byId(7).executeAsOneOrNull()
```

Fungsi

Hasilnya

executeAsList()

Semua baris

executeAsOneOrNull()

Satu baris, atau null

asFlow()

Flow yang ikut berubah,
slide berikut

Query yang diamati dengan Flow

commonMain/data/SumberLaporanOffline.kt

```
import app.cash.sqldelight.coroutines.asFlow
import app.cash.sqldelight.coroutines.mapToList

fun pantauSemua(): Flow<List<Laporan>> =
    db.laporanQueries.semua()
        .asFlow()
        .mapToList(Dispatchers.Default)
        .map { daftar → daftar.map { it.keLaporan() } }
```

`asFlow()` memancarkan hasil query, lalu memancarkan lagi setiap kali isi tabelnya berubah.

`keLaporan()` mengubah `LaporanEntity` menjadi `Laporan` dengan `StatusLaporan`, seperti DTO minggu lalu.

Bagian 3

Offline-first

Database menjadi satu sumber kebenaran, jaringan hanya menyegarkan isinya.

Siapa membaca apa

	Minggu 6	Offline-first
Layar membaca	Hasil panggilan jaringan	Query database lewat Flow
Jaringan	Satu-satunya sumber data	Menyegarkan isi database
Saat offline	Layar Gagal, daftar kosong	Data terakhir tetap tampil, ada pesan kecil
Draf form	Hanya di memori ViewModel	Disimpan otomatis ke tabel Draf

Baca lokal, segarkan dari server

commonMain/data/SumberLaporanOffline.kt

```
class SumberLaporanOffline(  
    private val db: LaporkampusDb,  
    private val api: LaporanApi  
) {  
    fun pantauSemua(): Flow<List<Laporan>> = /* slide tadi */  
  
    suspend fun segarkan() {  
        val dariServer = api.semua()           // bisa gagal  
        db.transaction {  
            db.laporanQueries.hapusSemua()  
            dariServer.forEach {  
                db.laporanQueries.simpan(it.id.toLong(),  
                    it.judul, it.lokasi, it.status, it.petugas)  
            }  
        }  
    }  
}
```

Kalau `api.semua()` gagal, exception keluar sebelum database disentuh. Data lama aman.

`transaction` membuat hapus dan simpan terjadi utuh. Flow hanya memancarkan sekali di akhir.

ViewModel: daftar dari database

commonMain/ui/DaftarViewModel.kt

```
class DaftarViewModel(  
    private val sumber: SumberLaporanOffline  
) : ViewModel() {  
    val laporan = sumber.pantauSemua()  
        .stateIn(viewModelScope,  
            SharingStarted.WhileSubscribed(5000),  
            emptyList())  
  
    private val _pesan =  
        MutableStateFlow<String?>(null)  
    val pesan = _pesan.asStateFlow()  
  
    init { segarkan() }  
}
```

Isi fungsi segarkan()

```
fun segarkan() {  
    viewModelScope.launch {  
        _pesan.value = try {  
            sumber.segarkan(); null  
        } catch (e: CancellationException) {  
            throw e  
        } catch (e: Exception) {  
            "Offline. Menampilkan data terakhir."  
        }  
    }  
}
```

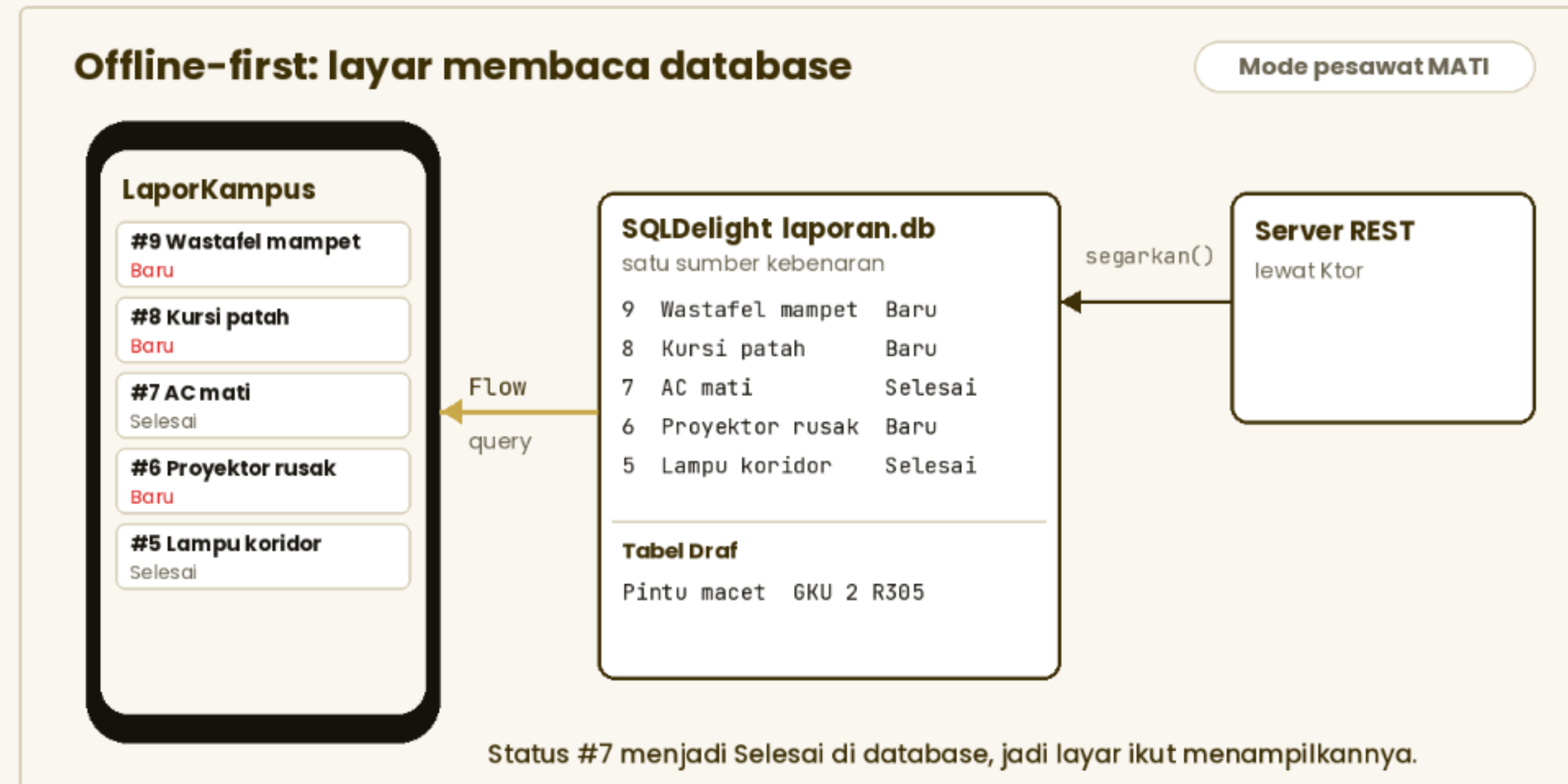
laporan selalu dari database. pesan hanya hasil segarkan, null bila berhasil. Tombol Coba lagi cukup memanggil segarkan().

BANGUN BARENG

Mode pesawat menyala, aplikasi ditutup

Lalu LaporKampus dibuka lagi. Apa yang tampil di daftar? Dan kapan laporan baru dari petugas muncul?

Cek tebakanmu: mode pesawat



Animasi berputar saat presentasi. Di PDF yang tampil keadaan akhirnya.

COBA LANGSUNG



Scan atau klik, tekan Run, lalu ganti jawaban ketiga menjadi null. Apa yang masih tampil di layar?

[Jalankan simulasinya di Kotlin Playground](#)

Sumber: Kotlin Playground, JetBrains. MutableStateFlow berperan sebagai database yang diamati layar.

Draf form tersimpan otomatis

commonMain/sqlDelight/.../Draf.sql

```
CREATE TABLE Draf (  
    id INTEGER NOT NULL PRIMARY KEY,  
    judul TEXT NOT NULL,  
    lokasi TEXT NOT NULL  
);  
  
simpanDraf:  
INSERT OR REPLACE INTO Draf  
VALUES (1, ?, ?);  
  
ambilDraf:  
SELECT * FROM Draf WHERE id = 1;  
  
hapusDraf:  
DELETE FROM Draf;
```

commonMain/ui/FormLaporanViewModel.kt

```
// di dalam FormLaporanViewModel  
init {  
    q.ambilDraf().executeAsOneOrNull()?.let { d →  
        _uiState.update {  
            it.copy(judul = d.judul, lokasi = d.lokasi)  
        }  
    }  
    viewModelScope.launch {  
        _uiState.debounce(500).collect {  
            q.simpanDraf(it.judul, it.lokasi)  
        }  
    }  
}  
// setelah kirim berhasil: q.hapusDraf()
```

Bagian 4

Coba sendiri, lalu bawa ke proyekmu

Tiga latihan SQLDelight dengan observasi praktik, data offline untuk tema kelompok, dan checkpoint proyek.

Yang saya amati selama latihan hari ini

Indikator RPS: **Data tersimpan dan dapat diakses offline**. Diamati dari latihan 2 dan 3, Repository dan OfflineNotes.

Kriteria	Sangat baik 85 sampai 100	Baik 70 sampai 84	Cukup 55 sampai 69	Kurang di bawah 55
Tersimpan dan bisa diakses offline CPMK0501, 4%	Catatan bertahan setelah aplikasi ditutup dan tetap tampil dalam mode pesawat	Tersimpan, hapus atau tampilan daftar belum benar	Data hanya di memori, hilang saat aplikasi ditutup	Belum ada penyimpanan lokal

Individu, bobot 4% nilai akhir. Bukti: push ke repository latihan pribadimu, lalu kirim tautan commit di LMS sebelum kelas selesai.

COBA SENDIRI

Tiga latihan di repo mata kuliah



Latihan 1

Handson1Preferensi.kt

Jumlah aplikasi dibuka yang tetap lanjut setelah aplikasi ditutup.

Latihan 2

Handson2Repository.kt

CatatanRepository dengan getAll sebagai Flow, tambah, dan hapus.

Latihan 3

Handson3OfflineNotes.kt

LazyColumn catatan, form tambah, dan tombol hapus yang datanya bertahan.

Semuanya di folder **P7 Local Data Storage Hands-on**, tanpa internet. Driver desktop di sana memakai memori, jadi uji data bertahan di Android.

Kalau error

Gejala atau pesan	Penyebab	Yang dicoba
Unresolved reference: AppDatabase atau CatatanQueries	Kode SQLDelight belum dibuat	Sync ulang Gradle, atau jalankan task generate
Data kosong lagi setiap dibuka di desktop	Driver desktop memakai IN_MEMORY	JdbcSqliteDriver dengan berkas jdbc:sqlite
no such table: Catatan	Skema belum dibuat di driver desktop	Panggil AppDatabase.Schema.create(driver)
NotImplementedError: Lengkapi getAll()	Latihan 2 belum selesai	Selesaikan latihan 2 dulu
Unresolved reference: asFlow	Dependency atau import belum ada	coroutines-extensions dan import-nya

Offline untuk tema kelompokmu

1 Skema .sq
Tabel entitas utama dengan query semua, byId, simpan, dan hapusSemua.

2 Driver per platform
expect dan actual DriverFactory untuk Android dan iOS atau desktop.

3 Sumber offline
Layar mengamati database, segarkan menulis hasil jaringan ke database.

4 Draf form
Tabel Draf yang terisi otomatis dan kembali saat form dibuka.

CONTOH DARI LAPORKAMPUS

Laporan.sql dan Draf.sql, DriverFactory, SumberLaporanOffline dengan pantauSemua dan segarkan. Mode pesawat menampilkan daftar terakhir.

Checkpoint proyek minggu 7

1 SQLDelight terpasang
Skema .sq untuk data utama dan driver di setiap platform kelompok.

2 Data terakhir saat offline
Layar membaca database, mode pesawat tetap menampilkan daftar.

3 Draf tidak hilang
Isi form tersimpan otomatis dan kembali setelah aplikasi dibuka ulang.

Dikumpulkan

Push ke repository kelompok. README memuat tangkapan layar daftar saat mode pesawat dan alur offline-first singkat.

Dicek di awal pertemuan berikutnya: tuntas, sebagian, atau belum.

TUTUP

Tiket keluar



- 1 Offline-first**
Kenapa layar membaca dari database, bukan langsung dari jaringan?
- 2 SQLDelight**
Apa yang kita tulis di berkas .sq, dan apa yang dibuatkan SQLDelight?
- 3 Pilihan**
Kapan cukup memakai preferensi, bukan SQLDelight?

Jawab singkat di kertas atau formulir LMS sebelum keluar kelas.

TUTUP

Minggu depan: fitur khusus platform



MASALAH MINGGU 8

Petugas fasilitas sulit mengecek laporan tanpa foto. Tombol Ambil foto bukti sudah ada di form, tetapi belum melakukan apa-apa. Bagaimana Laporkampus membuka kamera di Android dan iOS, dan apa yang terjadi kalau pengguna menolak izin?

Persiapan

Bawa HP Android dan kabelnya kalau ada, dan pastikan latihan SQLDelight sudah di-push ke repository latihan pribadi.

Terima kasih

Sampai jumpa di pertemuan 8

Ada pertanyaan? Sampaikan sekarang, lewat LMS, atau email ke muhammad.algifari@if.itera.ac.id